

Prom π t: AI-Powered Code Editor Architecture Specification

System Blueprint

June 16, 2026

Overview

This document outlines the foundational blueprint for **Prom π t**, an AI-powered code editor. By anchoring the product design around a modern web tech stack and integrating the mathematical concept of π (representing infinity and constant optimization), we establish a highly cohesive, performant, and extensible software architecture.

1 Core Technology Stack

The application will be packaged as a desktop executable using **Electron**, granting native OS capabilities while leveraging web technologies for the UI.

- **Frontend Framework (React):** Handles application state, multi-tab file workspaces, sidebar panels (AI chat, file tree), and settings interfaces.
- **Code Editing Engine (Monaco Editor):** The core of VS Code. Provides native IntelliSense, syntax highlighting, and text buffering.
- **Backend Runtime (Node.js via Electron Main Process):** Manages local file systems, native shell execution via Pseudo-Terminals (PTYs), and acts as the secure middleware for LLM orchestration.
- **AI Orchestration:** Uses chunked streams over IPC to feed code generations directly into the frontend without freezing the UI thread.

2 Proprietary AI Features (π -Themed)

- **π -lot Inline Autocomplete:** A contextual ghost-text engine that predicts the "infinite sequence" of your code logic based on real-time keystroke tracking.
- **The "Radius" Context Engine:** Automatically scans the codebase and gathers context from files that sit within a specific dependency "radius" of the active file.
- **Infinite Loop & Edge-Case Sentinel (∞):** A background compiler using structural AI analysis to catch infinite loops and memory leaks before execution.
- **"Radian" Multi-Agent Refactoring:** Spins up individual sub-agents to concurrently handle tests, typing, and business logic (representing a full 2π radians or 360° architectural perspective).

3 Language Support Architecture (Path B: Lean LSP)

To provide advanced language intelligence (like Python or C++ syntax validation) without the massive overhead of a full VS Code Extension Host, the editor utilizes the **Language Server Protocol (LSP)**.

- **Mechanism:** The Node.js backend natively spawns standalone Language Server binaries (e.g., `pylsp`, `tsserver`) on the user's operating system.
- **Bridge:** The React frontend uses `monaco-languageclient` to translate user keystrokes and file changes into LSP-compliant JSON-RPC messages.
- **Advantage:** Keeps the React UI incredibly fast and fully customizable for unique AI features, while seamlessly inheriting professional-grade code intelligence.

4 Electron IPC Communication Flow

To ensure the frontend and backend stay perfectly in sync securely, communication passes through a strict Three-Layer Topology:

1. **Layer A: The Secure Interface (`preload.js`):** A `contextBridge` that exposes a hardened API surface to React (e.g., `window.promptLsp`), preventing direct frontend access to the OS.
2. **Layer B: The Native Orchestrator (`main.js`):** Runs natively in Node.js. Manages language server binary lifecycles, file reading/writing, and network streams to LLM API providers.
3. **Layer C: The Client Wrapper (React + Monaco):** Hooks into Monaco's internal text models and dispatches structured document edit events to the backend securely.